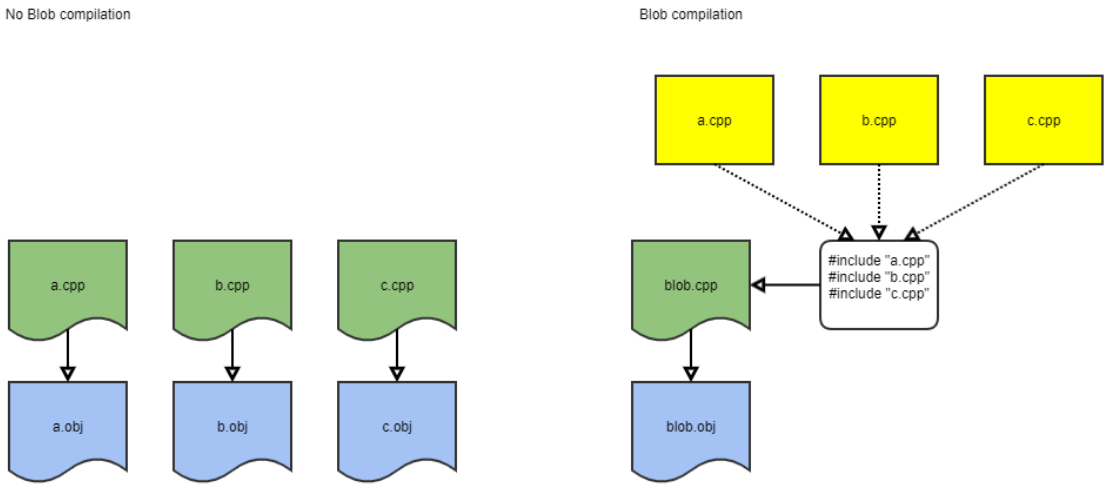
Programmation C++ avancéé – bases

Malek.Bengougam@gmail.com

Le système de génération (build system) du C++ est hérité de celui du C qui peut se résumer à cela :

A savoir un système conçu en 1972 sur un PDP-7 disposant de 144 KiB de mémoire.

Les possibilités de génération sont aujourd'hui multiples, et on va distinguer deux types de génération : une classique où chaque fichier est compilé en un objet distinct (à gauche) l'autre plus moderne et utilisée par des générateurs rapides et distribués fonctionnant par agrégation (*blob*, pour « *binary large object* », mais le terme est devenu plus générique et désigne tout amas de donnée).



Compilation en ligne de commande, exemple avec GCC

L'option '`-c <file>`' permet de spécifier le fichier source à compiler (les extensions sont généralement .c, .cpp). Cependant cette option ne s'arrête qu'à la génération d'un fichier objet et n'effectue pas le lien entre objet permettant de générer l'exécutable.

On peut modifier le nom du fichier objet à l'aide de '`-o <file>`' (extensions .o ou .obj selon le compilateur). A noter que si l'option '-c' est manquante '-o' donne donc le nom de l'exécutable final.

trivia : lorsque '-o' n'est pas spécifié, la génération produit un fichier exécutable nommé 'a.out' cf. https://fr.wikipedia.org/wiki/A.out

Attention, les options sont sensibles à la casse ! Par exemple '`-O` ' suivi d'un chiffre ou parfois d'une lettre permet d'indiquer le niveau d'optimisation souhaité.

'`-W<warn>`' permet de contrôler le niveau des avertissements de compilation, on utilise généralement 'Wall' où 'all' indique le niveau maximum.

'`-D`' permet de définir des macros (équivalent de #define) mais au moment de la compilation. Cela peut être utilise pour produire des options de compilations customisées.

'`-I<dir>`' spécifie un chemin vers un répertoire contenant des entêtes (includes, extension .h ou .hpp). Attention il n'y a pas d'espace après le '-I'. Chaque chemin doit être spécifié dans une entrée distincte.

'`-L<dir>`' spécifie un chemin vers un répertoire contenant des bibliothèques (libraries).

'`-l <lib>`' (en minuscule donc) indique le nom de la bibliothèque qui doit être liée au programme (extension .a ou .lib généralement). Si le nom de la bibliothèque commence par 'lib' il est alors possible de n'indiquer qu'une partie du nom dans la ligne de commande. Exemple : libpthread.a on spécifie '-lpthread'.

Note : pour créer une bibliothèque statique on doit utiliser le linker. Certains compilateurs comme GCC utilisent un outil spécifique : '.a' étant le diminutif de 'archive', l'outil se nomme **ar**.

'`-std=<version>`' permet d'indiquer la version du C ou C++ à utiliser lors de la compilation.

La plupart de ces options sont disponibles avec plus ou moins la même syntaxe sous d'autres compilateurs comme clang, Intel icc, Microsoft C/C++ cl -à noter que Visual Studio supporte également clang) etc…

Automatisation par makefile : CMAKE un outil de génération générique

CMake utilise un fichier CMakeLists.txt qui sert à contrôler la génération CMake peut être vu comme un générateur de configuration et de projet pour un compilateur ou un IDE.
CMake est une pierre dans le système de génération (build system) pour automatiser le déploiement d'un projet sous différentes configurations (machine, architecture, compilateur…) mais contrairement à make ce n'est pas un outil contrôlant explicitement la compilation.
Un utilisateur, après avoir défini le fichier de contrôle CMakeLists.txt, peut dès lors produire des fichiers spécifiques à un compilateur/IDE à l'aide de « generators »

Note : CMake génère par défaut un makefile utilisable directement par make.

Il est **fortement** recommandé de créer un répertoire 'build' au même niveau que le fichier CMakeLists.txt ce qui permet de séparer le résultat de la génération du code source.

```
mkdir build && cd build
cmake ..
```

Alternativement, on peut spécifier explicitement le répertoire de génération avec l'option -B

```
cmake – S . -B build
```

A ce stade il se passe pas mal de choses : CMake va détecter les compilateurs en présence, vérifier leur fonctionnement et, entre autres, définir des variables internes.
Par défaut, *cmake* génère un makefile, on peut donc directement appeler *make* (en étant dans le répertoire build) en tapant simplement 'make .'.

On peut spécifier un générateur en particulier via l'option -G. La liste des générateurs disponibles se trouve à cette URL https://cmake.org/cmake/help/latest/manual/cmake-generators.7.html .

Directives de base d'un CMakeLists.txt

L'ordre des opérations est important mais pas toujours primordial. Idéalement on se borner à suivre l'ordre de la génération d'un compilateur par ex, définition des répertoires (includes et libs), spécification des fichiers .cpp et finalement liste des bibliothèques à lier.

Avant cela il faut définir quelle version de cmake est requise pour que notre projet puisse être généré, et, très important, définir le nom du projet ainsi que la version attendue du compilateur.

```
#numéros de version Majeur et mineur
cmake_minimum_required(VERSION M.mm)
#nom du projet qui est egalement le nom de l'executable
project(<nom_executable>)
#quel type de compilateur (ici C++) ainsi que la version supportée (ici C++14)
set(CMAKE_CXX_STANDARD 14)
```

includes_directories() et **link_directories()** permettent respectivement d'indiquer les chemins vers les répertoires d'entêtes et de bibliothèques.

Finalement on peut spécifier l'ensemble des fichiers cpp à compiler pour un projet spécifique

```
add_executable(<nom_du_projet> <premier_fichier.cpp> <deuxieme_.cpp> …)
```

Si le programme est dépendant de bibliothèques alors il faut également détailler la liste avec

```
target_link_libraries(<nom_du_projet> <premiere_lib> <deuxieme_lib> …)
```